Architectural Patterns for Integrating LLMs into User-Facing Applications

Lessons from a language-learning platform

MIRCEA LUNGU, IT University of Copenhagen, Denmark

CESARE PAUTASSO, University of Lugano, Switzerland

An LLM placed behind a feature in an existing application has to meet expectations it was not designed for: an answer in the time a user will wait, at a cost the feature can carry, in a form the surrounding code can consume. How to reconcile a slow, costly and non-deterministic model with those expectations is little documented. Products built around the model, such as chatbots, agents and retrieval-augmented pipelines, have taken most of the attention. Fitting a model into an application that already exists is a different problem. We present a catalog of nine recurring architectural patterns for that case, grounded in a year and a half of LLM integration work on Zeeguu, an open-source language-learning platform with several hundred monthly active users. The patterns are grouped into three themes, using the LLM efficiently, trusting its output, and managing change over time, and described in the standard context, forces, solution, and consequences format. Each pattern names the Zeeguu features it came from and what corroborates it elsewhere, whether published work, a tool that ships the same mechanism, or another team's engineering account, and says where we found no such use outside Zeeguu.

Additional Key Words and Phrases: large language models, software architecture, design patterns, LLM integration, cost, latency, quality assurance

1 Introduction

While there is growing literature on building products whose central function *is* the model — chatbots, agents, and retrieval-augmented generation (RAG) pipelines, which practitioners often call *LLM-native* — and on using LLMs for code generation, there is surprisingly little guidance on the **software engineering challenges of integrating LLMs as components into existing interactive applications** where real users expect fast, reliable, and trustworthy responses.

We expect this integration to become the common case: over time, more and more existing user-facing applications will adopt LLMs not as standalone chatbots but as components working behind the scenes to improve the user experience. This is why we present a set of patterns that highlight the challenges and opportunities such integrations raise.

Indeed, LLMs have a unique combination of properties:

- **expensive**: billed per token, with a large fixed prompt re-paid on every call;
- **slow**: responses take seconds, not the milliseconds an interactive interface expects;
- **non-deterministic**: the same input can return a different answer;
- **rapidly evolving**: models are released and retired on the vendor's schedule, every few months;
- **imprecise**: they make mistakes, from a malformed response to a confident but wrong answer; and
- **general-purpose**: they can attempt almost any task expressed in text.

Putting a component with these properties into a live, user-facing system pulls a design in two directions at once: the model is slow, costly and non-deterministic, while users and the surrounding code need answers that are fast,

Authors' Contact Information: Mircea Lungu, IT University of Copenhagen, Copenhagen, Denmark, mircea.lungu@gmail.com; Cesare Pautasso, University of Lugano, Lugano, Switzerland.

affordable and well-formed. Competing pressures of this kind are what the pattern literature calls **forces**, and every pattern in this paper is an account of which forces it resolves and at what cost.

Over the past year, we have been integrating LLMs into Zeeguu[1], an open-source platform for personalized language learning that helps users learn foreign languages by enabling them to read authentic online content (real articles, not textbook exercises). Through this work, we have identified a set of architectural patterns for LLM integration, several of them corroborated by documented use in other systems. Because these pressures come from the LLM itself and from the ordinary demands of a live application, not from anything specific to language learning, we expect them, and the patterns that resolve them, to recur wherever LLMs are integrated into user-facing systems.

The remainder of the paper is organized as follows. Section 2 describes how the patterns were identified and how the resulting claims were checked. Section 3 introduces Zeeguu, the platform that grounds every example. The catalog then follows in three themes, using the LLM efficiently (Section 4), trusting its output (Section 5), and managing change over time (Section 6), each pattern given in the same format: context, example, problem, forces, solution, consequences, known uses, notes. Section 7 asks what makes these patterns specific to LLMs, and the paper closes with related work, limitations, and conclusions.

2 Methodology

The patterns come from one production system. This section states how they were found, so that a reader can judge what the evidence supports.

The corpus. Zeeguu (Section 3) has been in continuous operation since 2017 and currently serves several hundred monthly active users across eleven languages. Between January 2025 and August 2026 we added LLM-backed functionality across roughly a dozen of its features. The patterns below draw on ten: article simplification, CEFR difficulty assessment and summarization, grammatical correction of simplified text, topic classification, multi-word expression detection, on-demand translation escalation, audio-lesson scripting, audio-lesson topic validation, exercise sentence generation, and exercise sentence validation. Those features, and the code and commit history behind them, are the raw material.

Where the patterns came from. We did not set out to mine patterns. The design decisions came first, made under ordinary delivery pressure as each feature was built. Only after a year and a half of this did we ask which of those decisions had been reusable: which solutions we had reached for more than once, and which we would reach for again in a different system. The patterns presented here are our answer to that question, and the conditions below are what we then required of each.

What earned a place here. Two conditions. Inside Zeeguu, a solution had to appear in at least two features that reached it independently, rather than one copying the other; those instances are named with each pattern, and the source is public, so a reader can check them. Outside Zeeguu, we looked for the same solution in another system, a practitioner report, or published work, and each pattern states the closest evidence we found and how close it is. Applying these conditions changed what this paper contains: solutions we had expected to present did not survive them.

The evidence base for known uses. Corroborating uses come disproportionately from grey literature: engineering blog posts, provider documentation, and open-source code, rather than peer-reviewed papers. This is deliberate, and at this point unavoidable. The practice is roughly three years old and largely industrial, and the peer-reviewed record has

[1] https://zeeguu.org

not caught up (Section 8). Garousi et al.'s criteria for admitting grey literature as evidence in software engineering are met here on several counts: the topic is not covered by the formal literature, practitioners are the population that holds the knowledge, and the field is moving faster than the publication cycle. We mark each known use with its source type, so a reader can weigh it.

What this is not. This is not a systematic literature review, and not a controlled study. It is one team's reading of one system in one domain. The claim we make is that the pressures a pattern resolves come from the LLM's own cost, latency and unreliability, and from what a live application owes its users, neither of which is specific to language learning. Whether the patterns hold elsewhere is an empirical question, and Section 9 returns to it.

3 Case Study: Zeeguu

Zeeguu is an open-source language learning platform built around the idea that learners benefit most from engaging with comprehensible [12] and authentic [8] content in their target language. Rather than relying on artificial textbook texts and exercises, Zeeguu helps users find real articles (news, blog posts, and other web content) tailored to both 1) their *level* and 2) their *interests*. Then, based on the words they don't understand, it generates personalized vocabulary exercises and audio lessons.

The platform recommends articles in the learner's target language based on their proficiency and reading preferences, making it **easy to find material that is both engaging and appropriately challenging**. If a text is personally compelling but too difficult, Zeeguu simplifies it to the learner's level using LLMs.

When users encounter unfamiliar words or phrases while reading, they can get contextual translations on the fly from several translation providers, so the reading experience remains fluid and uninterrupted. One alternative, available on demand, comes from a state-of-the-art LLM offering a more contextually nuanced option (the "Ask an AI" escalation described below).

Every translation a user requests is logged by the system, which over time builds a **detailed model of the learner's vocabulary knowledge**, tracking which words they know, which ones they struggle with, and how well they've retained previously encountered vocabulary.

Based on this evolving learner model, Zeeguu **generates interactive vocabulary exercises and audio lessons** that focus on the words that matter most for each individual learner, rather than following a generic curriculum. The exercises use the context in which the word was originally encountered, based on the assumption that if the original text was compelling to the learner, examples drawn from it will be too.

In essence, Zeeguu unifies reading, translation, learner modeling, and practice into a coherent pipeline, with the learner's own reading interests as the primary driver.

Zeeguu also supports teacher accounts: a teacher can assign texts to a class and follow each student's reading and exercises. Because a teacher's judgment is more authoritative than an individual learner's, their corrections act as a higher-trust signal in the learner model.

3.1 The Pieces

A handful of Zeeguu-specific concepts recur across the patterns. They are collected here once, so a pattern can point back to a single definition rather than re-explaining them.

3.1.1 CEFR Levels. The Common European Framework of Reference grades language proficiency on an ordered six-level scale, from A1 (beginner) to C2 (mastery). Zeeguu uses it in two directions: to estimate how hard an article is, and to simplify an article down to the level of a given learner.

3.1.2 Article Simplification. When an article is compelling but too hard, Zeeguu rewrites it with an LLM to easier CEFR levels, producing one simplified version per level below the original. It runs both on demand, when a reader opens an article that has not been simplified yet, and ahead of time, over the crawled feed for some articles deemed likely to appeal to readers.

3.1.3 Crawling. Zeeguu builds its article recommendations by crawling news sites and blogs multiple times per day; this steady feed of freshly crawled articles is where most ahead-of-time LLM work happens (CEFR assessment, simplification), off any reader's critical path. On demand, readers push their own content in: a browser extension sends any article to Zeeguu for study, and on mobile the system share sheet sends any web page to be made interactive and studied within the application.

3.1.4 Multi-Word Expressions. A multi-word expression is a group of words whose meaning is not the sum of its parts, for example *break the ice*. Zeeguu detects them so a learner can translate the phrase as a unit rather than word by word. Detection runs on a dependency parse (Stanza [17], an open-source NLP parser). An LLM is called for one narrower case: translating an expression whose parts are separated in the sentence.

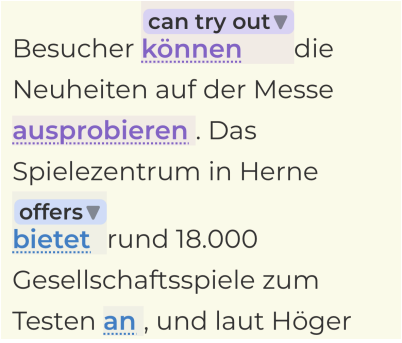


Fig. 1. Zeeguu's reader marks two multi-word expressions whose parts are apart in the sentence: können … ausprobieren (can try out) and bietet … an (offers). Both parts of an expression share one color and one translation. Stanza finds the expressions; an LLM translates them.

3.1.5 Meanings and The Learner Model. A *meaning* is a word paired with one particular translation of it. A meaning is linked to the original context in which it was seen, but can also be associated with other example sentences in which the word carries that same translation. Every translation a learner requests is logged as a meaning, building a model of which meanings they know and which they struggle with, since one word can have several meanings. Meanings are classified (by frequency, CEFR level, and phrase type: single word, collocation, idiom, or expression) and drive which exercises and lessons the learner sees. Vocabulary training trains *meanings*, not words.

3.1.6 Translation. While reading, a learner gets an instant contextual translation that is generated with the help of multiple parallel translation APIs. If they all agree, the translation is inserted above the word. If they disagree, a popup

surfaces the disagreement along with the option to escalate to an LLM through the "Ask an AI" action. This is one extra, more expensive step, offered for the cases where it is unclear which of the translations is correct.

3.1.7 Audio Lessons. Zeeguu generates personalized audio lessons: an LLM writes a short script built around the words a learner is studying, and text-to-speech synthesizes the audio. Both steps are slow and costly, so lessons are pre-computed for recently active learners. Once a learner chooses a topic (European history) or a situation (talking to an elderly neighbor on the stairway), a new lesson is generated each day, conditional on the learner having listened to the previous day's lesson.

3.1.8 Vocabulary Exercises. Exercises are built from the words a learner has looked up. They use example sentences drawn from the learner's own reading where possible, and LLM-generated, then LLM-validated, sentences otherwise. Original context is used only where possible, because not all contexts in which a learner encountered a word suit an exercise: some are too long, others do not make sense outside their original context.

3.2 Reach

Zeeguu currently serves over 300 monthly active users across 11 languages (Danish, Dutch, English, French, German, Greek, Italian, Portuguese, Romanian, Spanish, and Swedish), with peaks exceeding 400 during the academic year[2]. It is publicly available to try: the web app is at zeeguu.org[3] with the invite code `zeeguu-beta`, and the mobile apps can be downloaded from the iOS and Android app stores.

[2]Monthly active users are defined as users with any learning activity (exercises, reading, browsing, audio lessons, or translations) in a given month. Live statistics are available at: https://api.zeeguu.org/stats/monthly_active_users
[3]https://zeeguu.org

4 Using the LLM Efficiently

An LLM call is slow and metered, so a recurring question in any integration is how *not* to make the call, or how to make each one count. The patterns in this section keep the model's cost and latency off the user's path: paying a large fixed prompt once across a batch rather than once per item, reaching for the LLM only when a cheaper tool falls short, letting a classical stage gate it so it runs only where its judgment is needed, and computing likely-needed results ahead of time so the model never runs while a user waits. The unifying force is that the LLM is the expensive, slow component and the cheapest call is the one that is never made.

4.1 Prompt Amortization EFFICIENCY PATTERN

Batch many items into one call, so the fixed prompt is paid once.

4.1.1 Context. Many LLM calls share the same shape: a large, fixed instructional prompt (rules, output format, examples) wrapped around a tiny variable input, and re-sent in full on every call.

The work is offline and nobody is blocked on any single result, so an item can wait for others to join it.

4.1.2 Problem. Many calls send a prompt far bigger than the item it carries, and send it again for every item, so most of what is paid for is the same instructions, repeated. How can a system keep its detailed prompt without paying for it once per item?

4.1.3 Problem in Zeeguu. Zeeguu makes twenty-one kinds of LLM call. In most of them, a fixed template of a few hundred to several thousand characters wraps an input many times smaller (Figure 1). The translation-validation prompt[4], for example, wraps 5,096 characters of rules around a word, its translation and the sentence it came from. The templates also keep growing as rules are added: all four in the figure are longer than their first version (marked by the vertical tick).

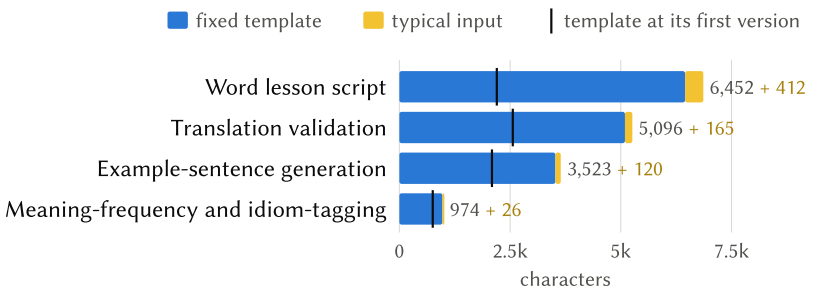


Fig. 2. Fixed prompt template (blue) and typical input (yellow), in characters, for four of Zeeguu's LLM calls at commit c853d76 (October 2026). The black tick marks each template's size at its first version.

4.1.4 Forces.

- **Detailed prompts work better.** The more precisely the rules describe the task, the more accurate and consistent the output. *(pulls toward big prompts)*
- **Cost follows the tokens sent.** Providers charge per token, and locally hosted LLMs use energy in proportion to the tokens they process. Every call sends its whole prompt again, so a big prompt is paid for once per item. *(pulls toward sending fewer tokens)*
- **Quality drops with more items per call.** The more items one call handles, the worse the model can handle each one: it might skip some, mix them up, or shorten its answers. With many small inputs this starts after about 15 to 20 items, long before the context window is full; with full-length outputs, after only a few. How much depends on the model and the task, and only measuring tells. *(pulls toward small calls)*

[4] https://github.com/zeeguu/api/blob/c853d767/zeeguu/core/llm_services/prompts/translation_validator.py#L8-L94

4.1.5 Solution. Put several items into one call, so the prompt is paid once per call instead of once per item,[5] and stop adding items before the quality of the answers drops. There are two ways to do it:

- ***Fan-in* batching** packs many independent inputs into one prompt, spreading a large instructional preamble across the whole batch.
- ***Fan-out* batching** produces many outputs from a single input, emitting one section per output and collapsing several requests into one.

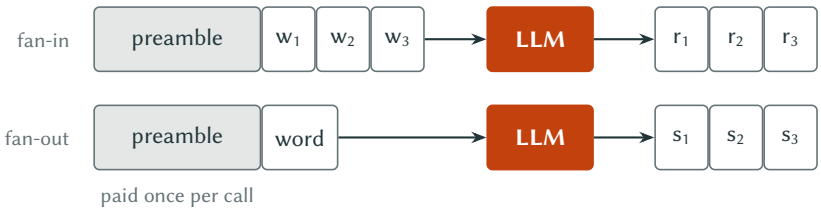


Fig. 3. Fan-in packs many inputs behind one preamble; fan-out asks for many outputs (here, several example sentences for one word) from one. Either way the preamble is paid once per call.

4.1.6 Solution in Zeeguu. Zeeguu uses both forms, on two of the calls in Figure 1:

- **Fan-in: meaning-frequency and idiom-tagging.** Zeeguu stores each word a learner saves together with its translation, a pair it calls a *meaning* (Section 3.1.5). An offline job sends about 15 of them per call and gets back for each one how common it is and whether the phrase is worth studying as a unit: an idiom such as *break the ice* is, a random pair such as *doctor for* is not.[6]
- **Fan-out: example-sentence generation.** One call asks for three example sentences for a word, so its template is paid once for all three.[7]

Fan-out also has a case where Zeeguu went back to one call per item. It used to ask for an article to be simplified (Section 3.1.2) to several levels in a single call: a B2 article came back at B1, A2 and A1 at once. Writing several versions in one answer, the model shortened them and merged paragraphs. An evaluation showed that one call per level simplifies much better, so Zeeguu now pays for the extra calls.[8] For that task, the batch size that works is one.

4.1.7 Consequences.

- **Cost and latency amortize with batch size.** The fixed preamble is paid once per call instead of once per item, so per-item token cost *and* wall-clock latency fall roughly inversely with the batch size.
- **A batch size to tune, and to tune again.** The size that works depends on the model, the prompt and the items, and only measuring finds it. Nothing signals when a new model or a longer prompt has moved it, so it has to be measured again after each change.

[5]Two provider features sound similar but do something else. *Batch APIs* (OpenAI Batch, Anthropic Message Batches) discount large asynchronous jobs, but every request in the job still pays for its full prompt. *Prompt caching* does discount a prompt that repeats across calls, but each item still makes its own call and waits for its own answer. It also works only under conditions: the cached prompt expires if no call reuses it soon enough (five minutes by default on Anthropic), and short prompts are not cached at all. As of October 2026 Anthropic's minimum is 512 tokens, about 2,000 characters, so the meaning-frequency and idiom-tagging template (about 1,000) is too short.

[6]The batched tagging prompt, `create_batch_meaning_frequency_and_type_prompt` in the `zeeguu/api` repository, run by `tools/reclassify_unclassified_phrases.py`.

[7]The example-generation prompt, `format_prompt` in the `zeeguu/api` repository, called with `count=3`.

[8]The old multi-level prompt, `get_adaptive_simplification_prompt`, is now reached only from a manual tool, `tools/simplify_article.py`. The split, its evaluation and its cost are in zeeguu/api#773.

- **Outputs have to be matched back to their items.** A batched answer can skip an item or stop short, so the code checks which items got a usable output, rather than assuming they line up, and sends the rest again (*Defensive Output Parsing*).
- **Fan-in adds a wait.** Items sit until a batch fills, so each result arrives later than it would on its own. This is why the pattern needs work that nobody is waiting on.

4.1.8 Known Uses.

- *The fan-in direction is prior art.* Packing many independent inputs under one shared prompt is the published **batch prompting [3]** technique (Cheng, Kasai & Yu, EMNLP 2023), which cuts token and time cost roughly inverse-linearly with batch size. This pattern's contribution is to add the *fan-out* direction and to frame both as a single move: amortizing one fixed preamble.
- The *fan-out* direction maps directly to structured-output calls that emit several keyed results at once, and to the OpenAI/Anthropic multi-output `n` parameter (which requests several completions per call).

4.1.9 Related Patterns. *Anticipatory Precomputation* (computing likely-needed results ahead of time) makes work offline, and offline work can wait for a batch to fill. *Task Fusion* is the mirror image: it shares one input across many tasks, where this pattern shares one set of instructions across many items.

4.2 Escalate to the LLM EFFICIENCY PATTERN

Serve the common case with a cheap tool; call the LLM only when the user questions the result.

4.2.1 Context. A feature can be produced either by a cheap specialized tool (a translation API, a classical classifier) or by an LLM. The tool's answer is shown to the user. The tool gives no score saying how sure it is, but the user often notices when its answer feels off.

4.2.2 Problem. Most results from the cheap tool are good enough, and paying for the LLM on those is waste. A few are not, and on those the LLM is worth its cost. How can a system call the LLM only for the results that fall short?

4.2.3 Problem in Zeeguu. In Zeeguu, translation APIs (from Google, Azure, and DeepL) translate the words a reader taps. Most of the time the translation is enough. Sometimes the reader is not convinced: the gloss does not fit the sentence, or a compound or idiom seems to mean something else here.

4.2.4 Forces.

- **The LLM does better on the hard cases.** On the inputs where the cheap tool falls short, such as idioms and compounds, an LLM usually does better. *(pulls toward using the LLM)*
- **Every LLM call costs money and time.** An LLM call costs more than the cheap tool and is slower, so making it on every request would make the common case slower and costlier. *(pulls toward using the cheap tool)*

4.2.5 Solution. Show the cheap tool's result first, and give the user a way to ask for a better one. On that signal, call the LLM with the original input and, where it helps, the cheap tool's output and the user's feedback, so the escalation refines rather than restarts. The LLM may answer a different question than the tool did: Zeeguu's translation gives a word, the escalation explains it.

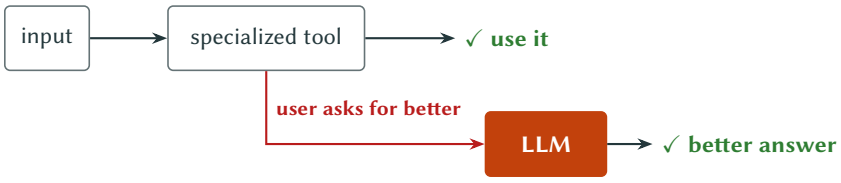


Fig. 4. The cheap tool serves the common case. The LLM runs only when the user asks for a better result.

4.2.6 Solution in Zeeguu. When the reader is not convinced by a translation, they tap *Explain* in the translation menu, and an LLM explains what the word means in this sentence, at the reader's level. The explanation works as a check: the reader can judge the translation instead of having to take it on trust.

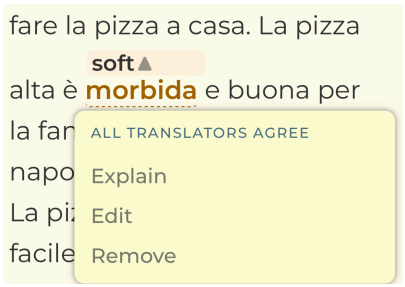


Fig. 5. In Zeeguu, the inline translation is the primary path; when the reader wants more, they escalate to an LLM on demand via the "Explain" option.

Explain does not send the LLM the translation the reader was shown, because the gloss misled it. For the Danish *narrøv* glossed as "foolish ass", the model reasoned backwards from the gloss and split the word into *nar* + *øv* ("donkey") rather than *nar* + *røv*. With the gloss in the prompt, 0 of 3 runs were correct; with only the word and its sentence, 3 of 3.[9] Whether the cheap tool's output helps the escalation or misleads it has to be measured.

4.2.7 Consequences.

- **Cheap in the common case.** The LLM runs only when a user asks, so most requests never incur its cost or latency.
- **The user carries the routing.** They must notice the option and use it, and it takes space in the interface. Some readers of earlier versions argued that the user should never have to ask; this pattern accepts that cost because nothing else can tell a poor result from a good one.
- **The better answer comes later.** The LLM starts only when the user asks, so its answer takes a full LLM call to arrive. The user is not left empty-handed meanwhile: they already have the cheap result.

4.2.8 Known Uses. **Duolingo Max**[10] (2023): after an exercise the learner sees the usual correction, and can tap *Explain My Answer* to get a GPT-4 explanation of why their answer was right or wrong. The explanation is generated only for learners who ask.

4.2.9 Related Patterns.

- **Model cascade.** In ML serving, a model cascade has the same shape, cheap tier first and expensive tier on demand, but escalates automatically when the cheap model's own confidence is low. **FrugalGPT [2]** (Chen, Zaharia & Zou, 2023) escalates when a scorer rejects the cheap answer, and **RouteLLM [16]** (Ong et al., 2024) trains a router that sends only the hard queries to the strong model. The difference is who decides: a score, or the user. Where the cheap tool gives a usable score, a cascade can decide without asking, and the two can be combined.
- ***Fail-Fast Provider Chain*** moves to another provider when a call fails. Here the cheap tool did not fail: its answer was not good enough for this reader.
- **Routing.** When the hard cases can be recognized before the cheap tool runs, send them straight to the LLM. Zeeguu does this for multi-word expressions (Section 3.1.4) whose parts are separated in the sentence, as *call*

[9] `explain_selection` and the measurements in `prompts/word_explanation.py`, in the `zeeguu/api` repository.
[10] https://investors.duolingo.com/news-releases/news-release-details/duolingo-max-shows-future-ai-education

and *back* are in *I'll call you back*: a translation API would get a fragment without its context, so the whole sentence goes to the LLM and the API is never called.[11]

[11] `translate_separated_mwe` in the `zeeguu/api` repository.

4.3 Anticipatory Precomputation EFFICIENCY PATTERN

Generate the LLM results users will likely need before they ask.

4.3.1 Context. A user-facing feature needs LLM results that are slow to compute, but what each user is likely to need next is predictable.

4.3.2 Problem. The user expects the result at once. How can a feature use LLM results without making the user wait for the model?

4.3.3 Problem in Zeeguu. The vocabulary exercises a learner practices are built from the words they looked up while reading, each paired with a machine translation. At reading time an imperfect translation is low-stakes: the reader is only making sense of the text, and can pick a better option from the alternatives menu. But once the platform selects a word for the learner to *learn*, they will drill that word-translation pair over many days, so its correctness suddenly matters: a learner should not drill a wrong translation. Checking with an LLM whether a word–translation pair is correct and worth learning takes seconds that an exercise screen would rather not wait for.

Generating an audio lesson (Section 3.1.7) takes much longer than checking a translation. The LLM writes a long lesson script, from a long prompt: so that a recurring topic, a "talking to a neighbor" lesson the learner has had before, comes back fresh rather than repeated, the prompt tells the model what the learner has already heard in past lessons. Text-to-speech then turns the script into audio. Together this takes many seconds, far more than a learner should wait.

4.3.4 Forces.

- **Users expect the result at once.** A real-time interface is expected to answer in about 200ms, and an LLM takes seconds. *(pulls toward having the answer before the user asks)*
- **Work nobody asks for is wasted.** A result made for a need that never comes is paid for and never used. *(pulls toward generating only on request)*
- **Results made early must be stored.** A result has to be kept from the moment it is generated to the moment it is asked for: bytes for a validated translation, megabytes for an audio lesson, for every user and every result ahead. *(pulls toward generating only on request)*

4.3.5 Solution. Predict what each user will need next, generate those LLM results offline (for example, in a scheduled job), and store them, so they are ready the moment the user asks. Keep an on-demand path for requests nobody predicted, so a miss degrades to a normal wait rather than a failure.

The prediction usually comes from one of three places:

- **A schedule the system controls.** When the system itself decides what comes next, such as which words a learner studies tomorrow, it knows the need in advance.
- **Habit.** A user who was active recently is likely to come back, so their next result is worth preparing.
- **Popularity.** Some requests recur across users, such as the most frequent search queries, so their results can be prepared once for everyone.

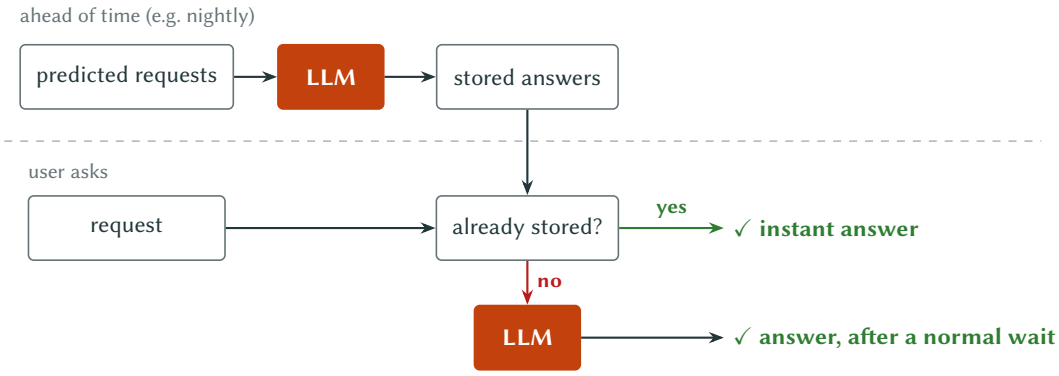


Fig. 6. Ahead of time, the LLM answers the requests the system predicts and the answers are stored. A predicted request is then served from the store with no LLM call; an unpredicted one falls back to an on-demand call, a normal wait rather than a failure.

4.3.6 Solution in Zeeguu. Zeeguu uses the first two sources:

- **Schedule: validated translations.** A regular cron job finds the words each learner is due to study next and validates them with an LLM, so when the exercise module asks for new words, the vetted ones are ready straight away. If none are precomputed yet (the learner translated a few words and jumped straight into exercises), the validation runs on demand.
- **Habit: audio lessons.** A nightly job generates the next lessons for recently active learners, those who practiced most recently first, on the assumption that someone who has been studying will be back for the next one. When they return, the lesson is already waiting. Only a learner who switches to a new topic waits, while that day's first lesson on it is generated on demand.

4.3.7 Consequences.

- **No wait for the model.** The LLM's cost and its seconds of latency are paid off the user's path; when the user acts, a predicted result is already there.
- **Some of the work is wasted.** How much depends on the prediction: a schedule the system controls wastes little, while habit and popularity pay for every guess that misses.
- **Storage grows with how far ahead the system looks.** Precomputing for more users, or further ahead, keeps more results waiting.
- **Two paths to maintain.** The on-demand path stays for requests nobody predicted, so the feature is built for both.

4.3.8 Known Uses.

- **Yelp**[12] precomputes LLM query-understanding responses for high-frequency ("head") search queries into a key/value store, "caching (pre-computing) high-end LLM responses for only head queries", reaching 95% of traffic for review-highlight expansions, so the expensive model never runs on the critical path.

[12] https://engineeringblog.yelp.com/2025/02/search-query-understanding-with-LLMs.html

- **Instacart**[13]'s Intent Engine serves high-frequency search queries from a precomputed cache of LLM outputs, leaving only ~2% of queries to a real-time model.

4.3.9 Related Patterns. *Hot-Path Result Caching* also serves stored LLM results, but stores an answer only after it was first requested; this pattern computes it before anyone asks. Yelp's own description, "caching (pre-computing)", sits between the two. Precomputed work is offline, so it can be batched with *Prompt Amortization*. The same move appears outside generation: **ColBERT [11]** (Khattab & Zaharia, SIGIR 2020) precomputes passage embeddings for a whole corpus, so a query runs only cheap matching.

[13] https://tech.instacart.com/building-the-intent-engine-how-instacart-is-revamping-query-understanding-with-llms-3ac8051ae7ac

5 Trusting LLM Output

An LLM's output is only probabilistically correct, and only probabilistically well-formed, yet it flows into code and data that assume it is neither wrong nor malformed. The patterns in this section guard that boundary: refusing to trust the format of a response, catching content errors with a second, narrower check, recording how far each stored piece of output has been trusted. The unifying force is non-determinism: the same prompt can return a different, or malformed, answer, so correctness has to be enforced around the model rather than assumed from it.

5.1 Defensive Output Parsing TRUSTING OUTPUT PATTERN

Check the format of every LLM response in code, and repair it if needed.

5.1.1 Context. An LLM is asked for output in a fixed format (JSON, or fields separated by a character such as `|`), and code reads that output. How much the LLM providers enforce such a format differs between providers and keeps changing.[14]

5.1.2 Problem. A response in the wrong format makes the code that reads it fail: either with an error the user sees, or, worse, by carrying on as if the response had said something it did not (treating an unreadable verdict as a yes, say). How can code use LLM output safely when some responses arrive malformed?

5.1.3 Problem in Zeeguu. Before a learner practices a word they saved while reading, Zeeguu asks an LLM whether the translation they saved with it is right, and code reads the verdict out of the reply. The reply used to be a single line: `VALID` followed by details about the word (how common it is, its CEFR level, A1 to C2, and its type), or `FIX` followed by the corrected word and translation. Two lines in that format, for a French word glossed in English (empty fields are allowed); the second corrects *campagne* from "campaign" to "countryside":

```
VALID|common|A1|single_word||
FIX|campagne|countryside|common|A2|single_word|||
```

In October 2026, Zeeguu moved from Sonnet 4.5 to Sonnet 5.5. The new model behaved differently with the same prompt: it wrote its reasoning before the `VALID|FIX` verdict line. On 22 of 50 production cases, it hit the 200-token limit before reaching the line. The parser found neither `VALID` nor `FIX`, and its rule for an unexpected reply was to assume valid. So 22 replies were passed, and almost all of them were translations that needed fixing.[15]

5.1.4 Forces.

- **Trusting the output is easy.** Most responses come back in the requested format, and more and more providers enforce it, so handing the response straight to the code is simple and usually works. *(pulls toward trusting the response)*
- **Models do not always follow the format.** Asked for JSON, a model may wrap it in a code fence, add a sentence first, stop halfway at the token limit, or refuse. *(pulls toward not trusting the response)*

5.1.5 Solution. Parse every LLM response inside a guard (a `try`/`except` around the parse) before any code uses it, and when the parse fails, fall back instead of raising. The fallback must count as a failure, never as a pass: skip the item, retry, or ask the next provider, and use a default only if it is the safe answer. On top of that minimum, add the following layers where they pay off:

- **Request the format.** Where the provider can enforce a schema, ask for it: formatting errors then disappear at the source.
- **Repair only known formatting errors.** Where the provider cannot enforce a schema, strip known wrappers (such as markdown code fences), attempt a plain parse, and if the model wrapped the output in prose, find the expected structure inside the text and parse that. Reject anything else rather than guess.

[14] OpenAI has enforced a requested JSON schema since August 2024 and Anthropic since November 2025. DeepSeek's JSON Output guarantees valid JSON but not a schema.
[15] Pull request #834 in the `zeeguu/api` repository, with the measurement on the 50 cases.

- **Check the content.** In both cases, check in code what no schema guarantees: that the response is complete, that it is not a refusal, and that it meets the constraints the code relies on (types, ranges, required fields, or that an index the model returns falls inside the input).
- **Make the fallbacks visible.** Log every response that fell back, with its raw text, and count them, so a change that breaks every response shows up as a jump in the count rather than as quietly emptier results.

Keep this strictness in code, where it is deterministic and testable, rather than in ever-longer prompt instructions. The question here is only whether code can use the output; whether usable output is *correct* is the concern of *Reject and Reprompt* and *LLM-Checking-LLM*.

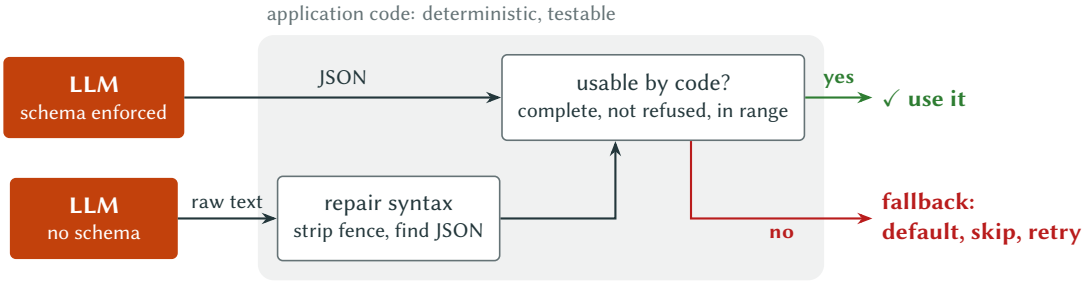


Fig. 7. A provider that enforces a schema sends JSON straight to the check; output from one that does not has its syntax repaired in code first. Either way, code checks what no schema guarantees (the response is complete, not a refusal, and any index it returns falls inside the input), and a response that fails degrades to a fallback instead of raising.

5.1.6 Solution in Zeeguu. The fix answers each part of that failure.[16] The validator now requests structured output: the API holds the reply to a JSON schema, so no free text can come before the verdict, and the schema's first field, `reason`, gives the model a place for the reasoning it wants to write. The model fills the fields in order, so the reasoning comes first and cannot crowd out the verdict. With the schema enforced, no repair layer is needed. The token limit went from 200 to 1000, so the reply is not cut off. And a reply that still cannot be read no longer counts as a pass: the word stays unchecked and the failure is reported. On the same 50 cases, no reply was wrongly passed. Simplified, the validator reads:[17]

```
def validate(word, translation, sentence):
    try:
        # a JSON schema with `reason` first; room for the whole reply
        reply = ask_llm(prompt, schema=VALIDATION_SCHEMA, max_tokens=1000)
        result = parse(reply)              # an unreadable reply raises
    except Exception as error:
        report(error)                      # shows up in error tracking
        return None                        # no verdict: the word stays unchecked
    return result
```

[16] Pull request #834 in the `zeeguu/api` repository, with the measurement on the 50 cases.
[17] `validate_and_classify`, `_ask` and `_parse_response` in `zeeguu/core/llm_services/translation_validator.py`, in the `zeeguu/api` repository.

5.1.7 Consequences.

- **A malformed response costs one result, not an error.** The layered parse recovers what it can, and the fallback covers the rest: at worst, one item is missing or, as in Zeeguu, one word stays unchecked.
- **The fallback rate has to be watched.** Users never see a malformed response, so only the log and the count show when a prompt or model change breaks them.
- **Repair can hide a broken reply.** The more the code repairs, the more it guesses: closing the brackets of a reply cut off at the token limit turns it into valid JSON that is silently missing its last items. Repairing only known formatting errors, and rejecting the rest, keeps the guessing small.
- **One parser per kind of call.** Each prompt asks for its own output format, so each needs its own parser and check, kept in step with the prompt as either changes. Zeeguu has about a dozen such functions.
- **The checks shrink as providers improve, but do not go away.** Schema enforcement removes the syntax repair, not cut-off answers, refusals, or constraints a schema cannot state, and a fallback model in the chain may enforce nothing. Waiting for the providers therefore does not remove the need for this pattern; it only makes the repair layer smaller.[18]

5.1.8 Known Uses.

- **G-Research**[19]'s code-review tool treats LLM output as "unverified input": it normalizes responses by "removing wrappers before parsing" (some providers return bare JSON, some wrap it in markdown fences), validates every finding against an authoritative rule index (invalid findings are "rejected outright"), detects truncation via the `length` finish reason (the API's signal that a reply was cut off) and retries with a lower cap, and sends structurally-invalid output back to the model for one repair pass.
- **Honeycomb**[20]'s Query Assistant parses the LLM output, "correct[s] it (if it's correctable)," validates it, and instruments parse vs. validation errors separately before running the query.
- *Prior art.* Finding the expected structure and skipping the text around it is the LLM-output version of island grammars [15] (parse the fragments of interest, skip the rest) and other error-tolerant parsing long used to pull structure out of irregular input.
- *Libraries.* Instructor[21] (Pydantic validation with automatic retry) and LangChain's `OutputFixingParser`[22] package parts of this pattern for reuse.

5.1.9 Related Patterns. A failed parse can trigger a retry, as in *Reject and Reprompt*, or the next provider, as in *Fail-Fast Provider Chain*. Provider schema enforcement taking over part of this pattern is the kind of drift the patterns in the section Managing Change Over Time address.

[18] Nor can a provider's guarantee be taken on trust: a provider can ship a regression like any other dependency, and DeepSeek's own documentation warns that its JSON mode "may occasionally return empty content". The checks in code stay as the last line of defense.
[19] https://www.gresearch.com/news/building-a-code-review-tool-the-llm-patterns-that-actually-work/
[20] https://www.honeycomb.io/blog/hard-stuff-nobody-talks-about-llm
[21] https://python.useinstructor.com/
[22] https://python.langchain.com/api_reference/langchain/output_parsers/langchain.output_parsers.fix.OutputFixingParser.html

5.2 LLM-Checking-LLM TRUSTING OUTPUT PATTERN

Check the output of an LLM with a second LLM call that asks a narrow question about it.

5.2.1 Context. An LLM generates content that will be used or stored, and its output can be wrong in a specific way. A narrow question about the result (is it grammatical? does it use the word in the intended sense? does it suit the learner's level?) can be answered on its own, without redoing the whole generation. No deterministic check can answer it, and there are too many outputs for a person to review each one.

5.2.2 Problem. How can an unreliable generator's mistakes be caught, when a second generator would be just as unreliable?

5.2.3 Problem in Zeeguu. The vocabulary exercises show a word in a sentence, with the learner's translation above it. For each word a learner studies, an LLM writes example sentences at the learner's level, and its prompt gives the word together with the translation being taught (its meaning (Section 3.1.5)). Even so, a generated sentence can use the word in a way the translation does not fit. Asked for a sentence with the Danish *selv* translated as *even* (as in *selv børn kan det*, "even children can do it"), the generator wrote *Selv om det regner, går vi en tur* ("even though it's raining, we're going for a walk"). The sentence is natural and contains the word, but there *selv* is part of *selv om* (*even though*) and does not mean *even* on its own. The exercise shows *even* above it, and a learner who practices with that sentence learns the wrong meaning.

That sentence was generated in October 2025. A year later it is the example of what not to do in the generation prompt, which now forbids using the word inside a larger expression. Such sentences became rarer, but did not stop. Other errors slip through the same way: a sentence that is not grammatical, or one that needs another form of the translation (*means* where the translation says *mean*).

5.2.4 Forces.

- **Another LLM call is no more trustworthy, and costs as much.** It can be wrong in the same ways as the first, and it adds a full call in tokens and latency to every item. *(pushes against a second call)*
- **Some checks are too general for a deterministic test.** No rule can tell whether a word is used in the intended sense: that takes understanding the sentence. *(pushes toward a model checking the output)*

5.2.5 Solution. Use one LLM call to generate a result, then a separate, differently-prompted LLM call to ask one narrow question about it.[23] The checker's prompt holds only what the question needs (the item and the question about it) and asks for a short verdict.

This escapes the paradox because a narrow question (*Is it grammatical?*, or *Does it use the intended meaning?*) has a small, well-defined answer space and a clear success criterion, so an LLM can do it more reliably than the open-ended generation that produced the output. Saunders et al. [19] measured this: models judge whether an answer is right more reliably than they produce a right answer.

[23] A separate call with its own prompt, not necessarily a different model.

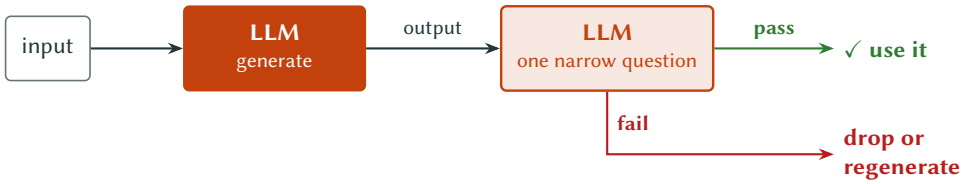


Fig. 8. A second LLM call asks one narrow question about the output of the first. An output that passes is used; what happens to one that fails is a separate choice: drop it or regenerate it.

A failed check still needs a policy, and the pattern does not fix one: drop the result, or regenerate it with *Reject and Reprompt*. Dropping is cheapest and fits where the item is expendable (as with Zeeguu's example sentences below). Regenerating with the reason the check gave, within a bounded number of attempts, is *Reject and Reprompt*, whose loop works with any checker, an LLM included.

5.2.6 Solution in Zeeguu. Before a generated sentence is used in an exercise, a second LLM call asks one narrow question: does this translation fit the word in this sentence? The check receives the word, the translation and the sentence, and works through a few steps that all serve that question: is the sentence grammatical, does the translation read correctly in place of the word and in the form the sentence needs, and is it the meaning of the word itself rather than of a larger expression around it. *Selv om det regner…* fails at the last step. A sentence that fails is dropped, and the exercise uses other sentences when they are available. And it works because answering whether one translation fits one sentence is far narrower than writing a good sentence from scratch.

5.2.7 Consequences.

- **Errors are caught for the price of one extra call per item.**
- **It narrows the error rate, it does not remove it.** The checker is itself an LLM and can return its own false verdicts. Generation and check can sit behind one call, so the rest of the application never sees the checker, but what comes out is still best-effort.

5.2.8 War Story: A Check Not Worth Its Cost. A check that works can still not be worth what it costs. Zeeguu ran a second call over every simplified article (Section 3.1.2): it read each simplified title and body and returned grammar corrections, which were applied to the text.[24] Between December 2025 and April 2026, roughly 96% of title corrections and 72% of body corrections changed two characters or fewer: small slips, not broken sentences. The check worked, but every simplified version paid an extra call for changes few readers would notice, and it is now disabled by default. That verification is cheaper than generation says nothing about whether the errors are worth catching; only measurement answers that.

5.2.9 Known Uses.

- *Documented in the literature.* **LLM-as-a-judge [25]** (Zheng et al., NeurIPS 2023) uses a separate strong LLM to score another model's open-ended outputs, now a standard evaluation technique.
- **Self-Refine [14]** (Madaan et al., NeurIPS 2023) has the model critique and refine *its own* output in a separate pass: a same-model variant of the idea, where this pattern instead uses a differently-prompted call.

[24] `GrammarCorrectionService` in the `zeeguu/api` repository.

- **G-Eval [13]** (Liu et al., 2023), shipped in eval frameworks such as **DeepEval**[25], uses a separate chain-of-thought (step-by-step reasoning) judge call to grade generated output.

5.2.10 Related Patterns. *Defensive Output Parsing* checks that an output is well-*formed*; this pattern checks that a well-formed output is *correct*. A failed check can feed *Reject and Reprompt*. Whatever happens on a failure, the verdict itself, pass or fail, can be stored with the output by *LLM Content Validation Tracking*, so later code knows what was checked.

The second call is not more generation. An ensemble averages several generations, and chain-of-thought elaborates one; this pattern spends the call on verification instead. Nor is it self-correction: a model asked to revise its own reasoning with no new signal shows no such gain and can even get worse (Huang et al. [10], Stechly et al. [21]). The check works because it asks a *different* question in a *separate* call, much as checking a proposed answer is often easier than finding one.

[25] https://deepeval.com/docs/metrics-llm-evals

5.3 LLM Content Validation Tracking TRUSTING OUTPUT PATTERN

Tag every stored LLM output with its validation state.

5.3.1 Context. An application stores what an LLM generates next to data people wrote or checked, and later features read both. Some of the generated data has been checked, some has not, and once stored the two look the same.

5.3.2 Problem. How can trusted and untrusted LLM-generated data be told apart after it lands in the database, so each use relies only on data checked enough for it?

5.3.3 Problem in Zeeguu. Zeeguu has an LLM rewrite each article it offers at easier reading levels (Section 3.1.2), and stores every version for the next reader who asks for that level. Most versions are fine, but some are not: the LLM sometimes writes a version in the wrong language, and readers find others confusing or broken. Once stored, a bad version looks like a good one, and it is served to every reader at that level.

5.3.4 Forces.

- **Checking before storing keeps the database clean.** If only output that passed a check is stored, everything in it can be trusted as it is. *(pushes toward checking before storing)*
- **Not every error can be caught before storing.** A cheap check can run on every output, but a thorough one, by another model or by a person, costs more than the system can spend on every item. And some errors only a reader notices, such as a text that is correct but confusing. *(pushes toward catching errors after storing)*

5.3.5 Solution. Maintain an explicit, queryable validation state for all LLM-generated content in the data model. Never let LLM-generated content silently become trusted data. Every query that reads this content filters on the states its use can tolerate. The validation state may be a simple flag, but more often it has several ordered levels (e.g., rejected, or one of the ordered levels unverified → passed an automatic check → used without complaint → confirmed by a trusted user).

The state records how far an item has been checked, not that it is right. A confirmed item can still be wrong, so a later signal must be able to lower its validation state.

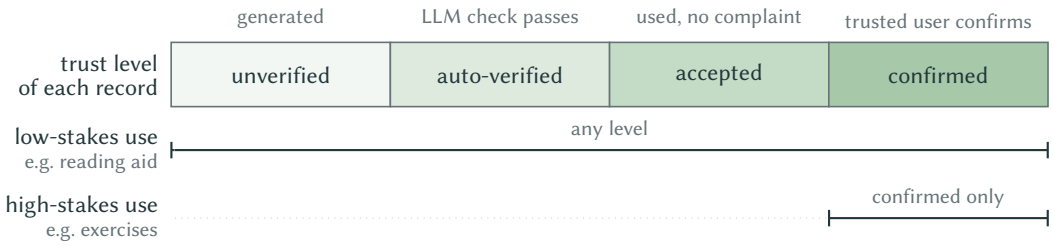


Fig. 9. Every stored LLM-generated record carries a validation state, raised or lowered by checks and by what users report. Each use reads only the states it can tolerate: a low-stakes use accepts any level, a high-stakes one only confirmed records.

5.3.6 Solution in Zeeguu. Each stored simplified article version carries a `broken` flag, together with the reason it was set.[26] Most versions in the wrong language are never stored: a language check runs when a version is written, and asks

[26] `Article.broken`, `UserArticleBrokenReport`, `available_simplified_versions`, `generate_in_language`, and `tools/audit_stored_article_languages.py` in the `zeeguu/api` repository.

the LLM to write it again (*Reject and Reprompt*). Two things set the flag. A script that checks the stored versions marks those that slipped through in the wrong language, and readers can report a version: two reports mark it broken, and so does a single report from a teacher, whose judgment the system trusts more. A broken version is no longer offered, and the next reader who asks for that level gets a newly generated one. This is the simplest form of the pattern: a single flag that only takes trust away, while versions nobody has checked are served by default.

5.3.7 Consequences.

- **Each feature decides how much checking it needs.** A feature where an error does real harm uses only output that a check or a person has confirmed; one where an error costs little can also use output nobody has checked yet. Either way, rejected output is never used as if it were good.
- **The data model carries a validation state to maintain.** It must be set on every write and updated on every check, and every feature that reads the content must filter on it: one that forgets serves rejected output. It can be a yes/no flag, several levels, or a count of the people who confirmed an item; which one fits depends on the domain.

5.3.8 Known Uses. The documented uses are data-labeling tools. We found no first-hand account of an application that tracks the trust state of its own LLM output, which is one reason we consider this pattern still emerging.

- **Argilla**[27], an open-source platform for annotating and reviewing model-generated data, gives every record an explicit status: a model suggestion starts as *pending*, becomes a *draft* and then *submitted* as someone works on it, and ends up *validated* or *discarded*. A suggestion is never treated as correct until a human has moved it to *validated*.
- **Label Studio**[28], another annotation tool, flags each item as ground truth or not and records whether it has been reviewed, so human-verified "gold" data stays clearly separate from unreviewed model predictions.
- **Snorkel [18]** (Ratner et al., VLDB 2017) attaches probabilistic confidence to programmatically generated labels rather than treating them as ground truth: a confidence score rather than fixed states.

The three tools show the range of options for how finely trust is recorded: a lifecycle with several states (Argilla), a yes/no "gold" flag (Label Studio), and a probability score (Snorkel).

5.3.9 Related Patterns. *LLM Output Provenance* records how an artifact was made; this pattern records whether it has been confirmed. Together they answer both questions about a stored artifact. *LLM-Checking-LLM* is one way to check an output, and its verdict is what this pattern stores.

[27] https://docs.argilla.io/latest/how_to_guides/annotate/
[28] https://docs.humansignal.com/guide/ground_truths

5.4 Reject and Reprompt TRUSTING OUTPUT PATTERN

Regenerate a rejected output instead of repairing it, and tell the model what was wrong. Record a refusal instead of retrying it.

5.4.1 Context. A check inspects each LLM output after it is generated. Code can fix some of the defects it finds. The check rejects the rest, and the only way to a correct result is to generate the output again.

5.4.2 Problem. How can a second attempt do better than the output the check rejected, and when should the system not try again at all?

5.4.3 Problem in Zeeguu. Zeeguu generates audio lessons (Section 3.1.7): dialogues in the language being learned, with a teacher who explains in the learner's own language. The model writes each lesson as a script that labels every line with its voice, and text-to-speech reads it out. An excerpt from a lesson that teaches *sobre* (*about*):

```
Teacher: In the following conversation, you will hear the word: [0.2 seconds]
TeacherL2: sobre [0.5 seconds]
Teacher: With the meaning [0.2 seconds]
Teacher: about. [1 seconds]

Man: ¿Qué piensas sobre este libro? [1 seconds]
Woman: Es muy bueno. Es un libro sobre animales. [1 seconds]
Man: ¿Sobre animales? Me gusta ese tema. [1 seconds]

Teacher: How would you say, what do you think about this book? [5 seconds]
Man: ¿Qué piensas sobre este libro? [3 seconds]
```

`TeacherL2` is the teacher speaking the language being learned, and the brackets are pauses. The prompt leaves the model room to write natural dialogue. It names the language, but cannot enforce it. In August 2026 a learner's daily lesson arrived with the whole script, dialogue included, in English instead of the language being learned. Nothing had checked it, and no code could have turned it into the right language.

That same month, five other lessons went wrong differently: the model did not write a dialogue at all.

5.4.4 Forces.

- **Every retry costs another call.** Each new attempt adds a generation and its latency, and an output that keeps failing can use up many calls. *(pushes toward giving up after the first rejection)*
- **A plain retry fails the same way.** Asking again with the same prompt draws from the same distribution as the first attempt, so the second output is about as likely to fail. *(pushes toward changing the prompt for a second attempt)*
- **Persistence pays.** Another attempt might succeed where the first one drifted. But some rejected replies are refusals the model was right to make. *(pushes toward telling refusals apart from bad outputs)*

5.4.5 Solution. Check each output after the LLM returns it. When the check rejects an output, regenerate it with a prompt that also says what was wrong, so the model knows what to avoid. The check can be cheap and need no LLM. The loop works the same with any check, including another LLM.

Bound the number of attempts. If they all fail, log the failure and go on without the output, when the application can do without it.

A refusal is different. It is a reply that is not an output at all: the model declines, or objects to the request, for example by pointing out a wrong input. It often comes back as text in place of the output, so the check must tell it apart from a bad output, for example by requiring a structure that a refusal lacks. Do not retry a refusal, not even with a note added to the prompt: the model will decline again, and it might be right to do so. Record it with the prompt version instead, so that the system does not ask again until someone changes the prompt, and so that a person can read why the model declined.

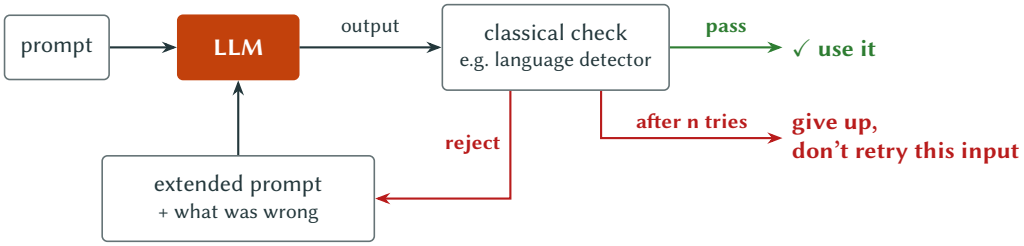


Fig. 10. A check sits downstream of the LLM. The system does not repair a rejected output. It adds what was wrong to the prompt and generates again. After a bounded number of tries the system gives up. If the model refused, the system records it and skips that input until the prompt changes.

5.4.6 Solution in Zeeguu. A language detector checks every generated audio lesson script. When it finds lines in the wrong language, the system generates the script again and adds a note to the prompt: *"Your previous attempt used the wrong language: [the lines in the wrong language]. Write it again. The Man, Woman and TeacherL2 lines MUST be in Spanish. The Teacher lines MUST be in English."* After three attempts the lesson fails for that night. Lessons are generated by a nightly job (*Anticipatory Precomputation*), so a failed lesson is tried again on the next run.

Each lesson teaches one word. Once the language check passes, a second loop asks an LLM whether the lesson teaches its word correctly. If not, the system generates the script once more and adds the LLM findings to the prompt.[29]

A refusal can arrive looking like output. Audio lesson scripts are free text, so a refusal is still valid text. The five lessons held a one-line refusal or a correction of the request instead of a dialogue. Text-to-speech had no dialogue lines to voice, so the lessons were silent. The check now requires lines in the voices that speak the language being learned (`Man`, `Woman` or `TeacherL2`), not only the teacher's. A reply without them is not a script. A dialogue in the wrong language still goes to the language loop.

In all five replies the model was right. Twice it pointed out that Zeeguu had supplied a wrong translation (*amenaza* means *threat*, not *hot*). Three times it declined words it was right to decline. Now a reply that is not a script ends the attempt. The system records it with the prompt version and generates no lesson for that word until the prompt changes or someone fixes the translation.[30]

5.4.7 Consequences.

- **The second attempt knows what went wrong.** Naming the failure usually works better than repeating the original instruction. The rest of the system only ever sees outputs that passed the check.

[29] `generate_script_in_languages` and `generate_lesson_script` in `zeeguu/core/audio_lessons/script_generator.py` in the `zeeguu/api` repository.
[30] `NotAScript` and `_is_a_script` in `zeeguu/core/audio_lessons/script_generator.py`, added in commit `8f8743f`, and `AudioLessonRefusal`, in the `zeeguu/api` repository.

- **The loop hides how often the model fails.** Every rejection costs another call and its latency, which is why the attempts are bounded. But retries that succeed look like success, so a model or prompt change that makes first attempts fail more often shows up only as extra calls. Track how often first attempts are rejected, per task, not only the final failures.
- **Refusals must be told apart from bad outputs.** A loop that retries a refusal pushes the model to produce what it declined to produce.

5.4.8 War Story: The Retry That Argued With a Refusal. The first fix for the silent lessons was a retry. When a reply came back as prose instead of a dialogue, the system asked again and added to the prompt: "*Do not explain, apologize, or comment on the request. Reply with the script only.*" The note treated a refusal as a formatting slip and tried to force the model past it. One of the words was related to child abuse, and the retry pressed the model to write the lesson it had declined. The model refused again, and it was right to. The current system respects the refusal and records it, so that the maintainers can see why the model declined.

5.4.9 Known Uses.

- **Instructor**[31] feeds the Pydantic validation error back into the prompt and asks the model again. This is the same loop, with a schema validator as the check.

5.4.10 Related Patterns. Defensive Output Parsing handles output that is malformed rather than wrong. It recovers what it can from the reply in hand, and fixes a stable, well-understood defect in place, with no second call. *LLM-Checking-LLM* spends a second LLM call on the check, and names this loop as one way to act on a failed check.

LLM Content Validation Tracking records the checks run on stored content. The refusal record does a different job: it keeps the system from attempting the work again. It matters most under *Anticipatory Precomputation*, where a scheduled job would otherwise rediscover the same refusal on every run.

[31] https://python.useinstructor.com/

6 Managing Change Over Time

An LLM integration is not static: prompts and models improve, vendors retire dated snapshots on their own schedule, and the LLM's own role in a feature shifts as the system matures. The patterns in this section manage that change: stamping stored output with the model and prompt that made it so the stale can be found and regenerated, retiring stale artifacts without breaking the past, and treating the LLM as a temporary stand-in for a cheaper component still to be built. The unifying force is a fast-moving, vendor-controlled substrate sitting under long-lived data.

6.1 LLM Output Provenance MANAGING CHANGE PATTERN

Tag every stored LLM artifact with the model and prompt that made it.

6.1.1 Context. LLM-generated artifacts are written to persistent storage and reused for a long time, while the models and prompts that produce them keep improving. The prompts live in the code, next to the calls that use them. The prompt and the model change independently of each other, and a change to either can have a large effect on output quality.

6.1.2 Problem. When a prompt or model improves, how can exactly the stale artifacts be found and regenerated, without reprocessing the entire store?

6.1.3 Problem in Zeeguu. When a learner saves a word, Zeeguu saves it in the form the learner met it in a text, for example the German *Rabatten* (discounts). Zeeguu then shows example sentences for the word, generated by an LLM and stored, so every learner of the word sees the same sentences.

The prompt, version `v3`, told the model to use the word exactly in that form, even where the sentence needed another one. So one stored sentence offered *hohe Rabatten* (high discounts), where German grammar needs *hohe Rabatte*. In October 2026 a new version of the prompt, `v4`, fixed this for new sentences.[32] But the store already held more than a hundred thousand sentences. Which of them came from `v3`?

6.1.4 Forces.

- **Regenerating everything is wasteful.** Most stored outputs would not change under the new prompt, and every regeneration costs a call. Some outputs have also been seen by users already, and would change under them. *(pushes toward regenerating only part of the store)*
- **The timestamp seems enough.** Every row records when it was created, and the code history says which prompt was deployed when. Recording more for every output costs schema changes and care on every write. *(pushes toward recording nothing more)*
- **The request logs exist already.** An LLM gateway or the provider's dashboard records every call with its prompt and model. Storing the same facts again next to every output looks like duplication. *(pushes toward relying on the logs)*
- **The deployed configuration is not always what ran.** A fallback (a backup model, called when the main one fails) can answer from another provider, and two prompts can be live at once during an experiment. *(pushes toward distrusting what the deployment history says)*
- **Prompts change by small edits.** Rewording a line in place is quick, and naming every edit as a new version feels like ceremony. *(pushes toward editing prompts in place)*

6.1.5 Solution. Store with every LLM-generated artifact a record of what produced it: the model, the prompt version, and when. This record is the artifact's *stamp*. Keep each (model, prompt version) pair once, in a shared generator table, and have every artifact point at its row. Request logs cannot replace the stamp: they are kept apart from the outputs, so a database query for stale outputs cannot see them.

Regeneration then becomes a query. It can be as broad as *"everything produced by prompt v2"*, or as narrow as one (model, prompt) pair. When a prompt does well on one model and poorly on another, only that combination is redone. The same query works across every kind of artifact at once.

[32] The `v4` prompt: `zeeguu/core/llm_services/prompts/example_generation.py`, commit `b8bd9eb0`, in the `zeeguu/api` repository.

Record the model that actually answered, not the one the configuration names. Use the same model identifier that the system uses to choose the model, kept in one place (*Centralized Model Selection*), so the stamp and the selection always name the same model.

Give each prompt a version name, and a new one whenever its text changes. A version in the prompt's file name, or a hash of the prompt text, makes this hard to forget. A stamp that stays the same while the prompt changes gives two different prompts the same name, and queries built on it return the wrong rows.

6.1.6 Solution in Zeeguu. Zeeguu keeps a table of generators, each a model name and a prompt version. Example sentences, simplified articles and audio lessons all point at it.[33] A single query found every sentence written by `v3`. Those that no learner was using were deleted, and new ones come from `v4` when a learner next needs them. Those in use stayed.

example_sentence

sentence	created_at	ai_generator_id
… hohe Rabatten …	2026-01-24	4
Wir sind gute Freunde.	2026-10-20	62
Morgen wird es regnen.	2026-10-20	62

ai_generator

id	model_name	prompt_version
4	deepseek-chat	v3
62	claude-sonnet-5-5	v4

Fig. 11. Provenance as Zeeguu stores it. Each example sentence keeps its own timestamp and points at a shared `ai_generator` row naming the model and prompt version that produced it. The sentence from prompt `v3` (red) is found by a query on that table.

Here the date alone would have worked too, because `v4` replaced `v3` on a single day. In general it is not enough. For more than a year, `v3` sentences were written by DeepSeek and by Claude at the same time, and the *Rabatten* sentence came from DeepSeek. Only the stamp tells them apart.

Not every artifact is stamped this well. Simplified articles record the model but not the prompt version, so a change to the simplification prompt cannot be traced in the stored articles.

6.1.7 Consequences.

- **Selective regeneration becomes a query.** Re-run everything a given prompt or model produced and leave the rest. The same record shows which model and prompt wrote what, for audits and for comparing them.
- **The guarantee lasts only as long as the discipline.** Every write path must stamp, including new kinds of artifacts added later, and every prompt edit must change the version. A stamp that drifts from what actually produced the artifact quietly returns the wrong rows.

6.1.8 War Story: The Nine Hundred Lessons. The audio lessons were not always stamped. Each lesson teaches one word. A teacher voice asks the learner to say something in German:

```
Teacher: How would you say you need at least ten minutes? [5 seconds]
```

But a learner who only hears the audio cannot know which of these to say:

[33] `AIGenerator` (`model_name`, `prompt_version`) in `zeeguu/core/model/ai_generator.py`, and the tables that reference it, in the `zeeguu/api` repository.

- *Ich brauche mindestens zehn Minuten* (I need at least ten minutes)
- *Du brauchst mindestens zehn Minuten* (you need at least ten minutes)

In May 2026 the prompt was fixed. But about nine hundred stored lessons already had questions like this one.

audio_lesson_meaning

id	the teacher's question, in the script	created_by
955	Say that you've changed departments.	claude-v1
3629	How would you say you need at least ten minutes?	claude-v1
4150	How would you say you don't like surprises?	claude-v1
37973	Say that you feel better after this cooling off.	claude-v1

the same label on every lesson, whatever prompt wrote it

Fig. 12. How Zeeguu stored audio lessons before the stamp. Every lesson carried the same label, whatever prompt had written it. The lessons with the ambiguous question (red) could only be found by their text.

Every lesson carried the same label, `claude-v1`, and the prompt file had been edited in place many times. Nothing said which lessons came from which prompt. The nine hundred were found only by searching the scripts' text for the ambiguous phrasing. A label that does not change when the prompt changes records nothing.

Care alone had not kept the versions honest, so the check moved into the build. A CI job now keeps a hash of the text of every versioned prompt. It fails when the text of a prompt changes but its version name stays the same.[34]

6.1.9 Known Uses.

- *Better attested in tooling than in production self-reports.* The capability is productized: MLflow Prompt Registry[35] and LangSmith[36] version prompts and record which prompt/model produced each output, confirming the pattern's core claim that the *prompt* deserves versioning as much as the model, but these are tools, not documented in-app deployments.
- *Adjacent production pipelines.* DoorDash[37] stores versioned LLM-generated profiles and "treat[s] prompts as code"; Etsy[38] stores Pydantic-validated LLM attribute extractions over 100M+ listings. Both store LLM artifacts at scale and version prompts, but neither publicly describes stamping *each artifact* with its prompt version to drive *selective* regeneration, which is the key mechanism here. We did not find a first-hand account of the full pattern, so Zeeguu is our instance.

6.1.10 Related Patterns. *LLM Content Validation Tracking* records whether an artifact has been confirmed. Provenance records how it was made. Together they say which artifacts to trust and which to regenerate.

Centralized Model Selection keeps the model identifier in one place. The stamp should use the same identifier.

Rent, Then Build trains a cheaper replacement on labels an LLM produced. Provenance lets it leave out the labels from a prompt version later found to be noisy or biased.

[34] `tools/check_prompt_versions.py` and `.github/workflows/prompt-versions.yml` in the `zeeguu/api` repository.
[35] https://mlflow.org/docs/latest/genai/prompt-registry/
[36] https://docs.langchain.com/langsmith/prompt-engineering-concepts
[37] https://careersatdoordash.com/blog/doordash-profile-generation-llms-understanding-consumers-merchants-and-items/
[38] https://www.etsy.com/codeascraft/understanding-etsyas-vast-inventory-with-llms

Soft Invalidation of LLM Artifacts takes over once provenance has found the stale outputs: it retires them from reuse while the old references that learners already heard keep working.

Reject and Reprompt records a refusal with the prompt version, so the system does not ask again until the prompt changes. That record relies on the same versioning of prompts.

6.2 Rent, Then Build MANAGING CHANGE PATTERN

Ship a feature on an LLM now, while building the cheaper component that will replace it.

6.2.1 Context. A feature needs to work in production now, but the efficient, dedicated version of it (a fine-tuned or classical model) needs training data or engineering that does not exist yet. A general-purpose LLM can do the task immediately, if expensively.

6.2.2 Examples. Zeeguu estimates the difficulty level of every article with an LLM, an expensive call it makes constantly. This is also the *bootstrapping* case: those LLM-generated difficulty labels are accumulating as the training set for a cheaper classical classifier that will take the task over once enough have been gathered. The LLM ships the feature and earns its keep today, while quietly producing the data that will one day retire it.

A plainer instance without the bootstrapping twist: article topic classification runs on an LLM now, to be replaced by a dedicated topic-detection model once the taxonomy of available topics settles.

The two tasks are waiting on different things, which is what determines how long each rental lasts. Difficulty assessment is waiting on *volume*, and every call it makes brings the replacement closer, so the stand-in shortens its own tenancy. Topic classification is waiting on a *decision* about the taxonomy, which no amount of LLM traffic advances. A rented capability that generates its own replacement has an end in sight; one that does not can run indefinitely without anything going visibly wrong.

6.2.3 Problem. How can a feature ship and start earning its keep before the cheaper, dedicated version that will eventually run it has been built?

6.2.4 Forces.

- **Shipping now captures the feature and its usage immediately.** A general-purpose LLM can do the task the day it is needed, before any dedicated version exists. *(pushes toward renting)*
- **Running the LLM indefinitely is costly and cedes control.** The feature keeps paying per call, and a core capability stays rented from a vendor who prices and deprecates on their own schedule. *(pushes toward building a replacement)*
- **The replacement cannot be built yet.** A fine-tuned or classical version needs training data or engineering that does not exist at the outset; only running the LLM produces it.

6.2.5 Solution. Use the LLM to perform a task in production while building a more efficient replacement. The arc is rent, then build: the LLM's general capability is *rented* (paid per call, costly but available immediately) to ship the feature now, while a cheaper, dedicated implementation is *built* to eventually own the task. The rented LLM is convincing to users and good for early beta-testing and feedback, but intended to be temporary.

Bootstrapping variant: In the strongest form, the LLM *generates the training data for its own replacement* (as with the difficulty labels above): the rented capability directly produces the labeled data the dedicated successor is trained on, so running the stand-in funds and enables the build.

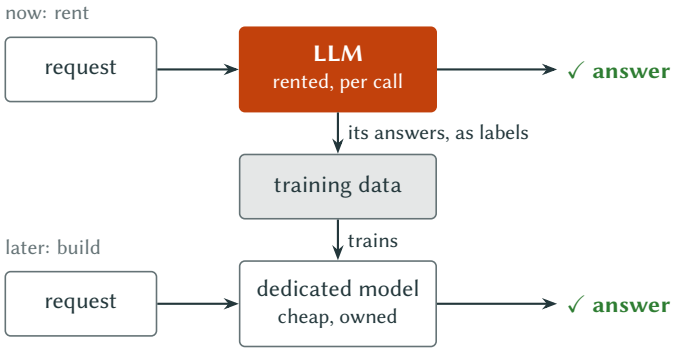


Fig. 13. Now, a rented LLM serves the feature, paid per call. Later, a cheaper dedicated model owned by the system takes the task over. In the bootstrapping variant drawn here, the LLM's answers become the training data for the model that replaces it.

6.2.6 *Consequences.*

- **The feature is live from day one.** It gathers real usage and feedback immediately, with the LLM standing in for a component that does not exist yet.
- **Running the stand-in funds its successor.** The LLM's inputs and outputs accumulate as the labeled data the dedicated replacement needs, so the temporary solution also produces the training set for the permanent one (the bootstrapping variant).
- **The stand-in is expensive, and "temporary" can stick.** Until the replacement ships, the feature runs at LLM cost and latency, the replacement is a second system to build and validate, and the intended-temporary LLM can quietly become permanent.

6.2.7 *Known Uses.*

- **OpenAI Model Distillation**[39] (Stored Completions) captures a large model's production input–output pairs and fine-tunes a smaller model as a drop-in replacement: the pattern shipped as a product feature.
- **Self-Instruct / Alpaca [22]** (Wang et al.; Taori et al., 2023) use a strong LLM to generate the instruction data that fine-tunes a small replacement: the bootstrapping variant.
- **Distilling Step-by-Step [9]** (Hsieh et al., ACL Findings 2023) trains task-specific models up to 700× smaller from LLM-generated rationales.

These are distillation and self-instruct methods from the literature; we did not find a first-hand account of a production feature completing the full rent-then-build arc, and Zeeguu's own replacement is still in progress.

6.2.8 *Notes.*

- This pattern has a lifecycle relationship with *Escalate to the LLM*: a system may start with the LLM as primary (renting it), migrate to a specialized tool as primary, and then keep the LLM as the escalation path, completing a full cycle from LLM-first to LLM-on-demand.

[39] https://openai.com/index/api-model-distillation/

- *Depends on LLM Output Provenance* for the bootstrapping variant: because the replacement trains on the rented LLM's own outputs, those labels must be filterable by the prompt and model version that produced them, so labels from a prompt version later found noisy or biased can be excluded from the training set.
- *Composes with LLM Content Validation Tracking* in the bootstrapping variant: gating the training set to validated outputs (confirmed, not merely generated) lets the dedicated successor learn from checked labels rather than the stand-in's unverified guesses.

7 What Makes These Patterns LLM-Specific?

Some of these patterns echo general distributed systems wisdom: batching (as in *Prompt Amortization*), tiered service, a cheap tier first and a costly one on demand (as in *Escalate to the LLM*), and the redundant dispatch of Zeeguu's parallel translation providers (not cataloged here as a pattern of its own). What makes them distinctly relevant to LLM integration is the combination of forces that arise when an LLM's properties meet the demands of a live system:

- **Cost structure**: per-token pricing with high fixed prompt overhead, unlike flat-rate API calls (drives *Prompt Amortization*, *Escalate to the LLM*, *Anticipatory Precomputation*).
- **Non-determinism**: the same input can yield a different, or malformed, output, so correctness must be enforced around the model (drives *Defensive Output Parsing*, *LLM-Checking-LLM*, *LLM Content Validation Tracking*).
- **Asymmetry between generation and verification**: answering one narrow question about an output is easier than producing it (drives *LLM-Checking-LLM*).
- **General-purpose capability**: the same component can serve as prototype, primary, or fallback (drives *Rent, Then Build* and *Escalate to the LLM*).
- **A rapidly evolving, vendor-controlled substrate**: models and prompts improve and are deprecated on the vendor's schedule, underneath long-lived data (drives *LLM Output Provenance*).
- **Quality-cost-latency tradeoff space**: uniquely wide compared to traditional APIs, and what the efficiency patterns navigate.

8 Related Work

To our knowledge, no peer-reviewed work presents a catalog of architectural patterns for integrating LLMs as components into existing production systems, grounded in real deployment experience and described using the standard pattern format (context, forces, solution, consequences). The contribution is the catalog itself, organized by three themes: using the LLM efficiently, trusting its output, and managing change over time. Some patterns apply established mechanisms (batching, precomputation, recall gates, soft-delete, distillation) to the distinctive forces of LLM integration; others, notably *LLM Output Provenance*, *LLM Content Validation Tracking*, and the external-signal trigger of *Escalate to the LLM*, appear to be new. We flag each honestly as one or the other throughout.

Why the gap exists. An absence is worth accounting for. Three things explain this one. The practice is young: production LLM integration became affordable and reliable enough for small teams only around 2023, so the systems this paper describes are roughly three years old, and a pattern needs several years of independent reinvention before anyone can recognize it as one. The knowledge sits with practitioners rather than researchers, and the systems that hold it are commercial, so what is written down appears as engineering blog posts and conference talks rather than papers, and the systems themselves are rarely open to inspection. And the academic attention that does exist has gone elsewhere: the large surveys treat LLMs as tools *for* software engineering, for code generation, repair and testing (Fan et al., 2023 [4]; Zhang et al., 2024 [24]), not as components inside the software being engineered. The one systematic review of software architecture and LLMs found 18 relevant papers in total and called the area open (Schmid et al., 2025 [20]).

This is also why our own corroborating evidence leans on grey literature, and why we think that is defensible rather than a weakness: Garousi et al. (2019) [6] argue for admitting it exactly where the formal literature is silent, practitioners hold the knowledge, and the field moves faster than the publication cycle. All three hold here.

Several practitioner-oriented resources discuss patterns for building LLM-based systems, but they operate at a different level of abstraction and lack the grounding in a specific production system that we aim to provide.

8.1 Practitioner resources

Eugene Yan's **Patterns for Building LLM-based Systems & Products [23]** (2023, blog post) identifies seven patterns: Evals, RAG, Fine-tuning, Caching, Guardrails, Defensive UX, and Collecting User Feedback. These address the question "how do I build an LLM product?" They describe the overall stack for LLM-native applications. Our patterns address a different question: "I have an existing system, and I want to add LLM capabilities as a component: how do I manage cost, quality, latency, and lifecycle?" There is some overlap on caching/pre-computation, but our treatment focuses on prompt amortization and user-need anticipation rather than semantic similarity caching. Yan's work is not peer-reviewed.

ThoughtWorks' **Emerging Patterns in Building GenAI Products [5]** (Fowler et al.) and "Engineering Practices for LLM Applications" focus on operational concerns: testing, evaluation, guardrails, and RAG pipelines. These are primarily about LLMOps rather than the architectural decisions for embedding LLMs as components within an existing application.

Andreessen Horowitz's **Emerging Architectures for LLM Applications [1]** (2023) provides a reference architecture for the LLM infrastructure stack (embedding pipelines, vector databases, orchestration layers, agents). Again, this targets LLM-native products rather than LLM integration into existing systems.

Books. "**LLM Design Patterns**" (Huang, Packt, 2024) and "LLMs in Enterprise" (Menshawy & Fahmy, Packt, 2025) cover model-level patterns (fine-tuning, quantization, inference optimization, RAG) and enterprise deployment concerns. They do not address application-level integration patterns such as the lifecycle management (Rent, Then Build → specialized tool → Escalate to the LLM), prompt amortization, or LLM output provenance that we identify.

8.2 Academic surveys

There is a large body of work on **using LLMs *for* software engineering tasks**: code generation, bug repair, testing, requirements engineering (see surveys by Fan et al., 2023 [4]; Zhang et al., 2024 [24]). However, these focus on LLMs as tools for developers, not on the engineering challenges of integrating LLMs as runtime components within production software.

A recent **systematic literature review on software architecture and LLMs** (Schmid et al., 2025 [20]) found only 18 relevant papers and noted that LLM-based software design remains an open research direction. None of the surveyed academic work addresses the specific architectural patterns for managing cost, quality, latency, and lifecycle when LLMs serve as components in existing applications.

LLM self-verification. For the *LLM-Checking-LLM* pattern, there is relevant work on **LLM self-verification**. Gero et al. (2023) [7] demonstrated that self-verification improves clinical information extraction accuracy, explicitly building on the asymmetry between verification and generation. However, Stechly et al. (2024) [21] showed that self-critique fails for formal reasoning tasks, finding significant performance collapse with self-verification but gains with external verification. Our pattern differs from both in that it uses separate, differently-prompted LLM calls for generation and verification of different properties (e.g., text simplification followed by grammatical correction), rather than asking an LLM to verify its own reasoning.

9 Limitations and Future Work

The patterns in this catalog have been extracted from a single production system, Zeeguu, in the language-learning domain. We have argued throughout that these forces arise from the LLM's own properties (per-token cost, multi-second latency, non-determinism, imprecision, a rapidly shifting provider landscape, and general-purpose capability) meeting the demands of a live application, neither of which is specific to language learning, and that the patterns should therefore generalize to other interactive applications that surface LLM-generated text to end users. Whether this is borne out in practice is an empirical question we cannot fully answer from a single case study. The most direct way to validate or refute the catalog is to gather complementary examples (and counter-examples) from practitioners working in other domains. A PLoP writers' workshop is one venue for exactly this kind of community refinement; mining open-source repositories for additional instances of these patterns (and patterns we missed) is a natural follow-up.

A further threat is to the premise rather than to the patterns. We expect LLMs to sit behind existing features more and more often, but assistants running on dedicated hardware could instead become where people go for everything, reducing applications to interfaces those assistants call. On that reading this paper describes a transitional form. We do not believe this will happen very soon but if it does, indeed, the relevance of these patterns might decrease.

A second limitation is that the catalog reports patterns we have found useful but does not yet quantify their impact systematically. Cost and latency improvements are reported informally for some patterns; a more rigorous deployment-level evaluation (measuring, for example, how much of Zeeguu's LLM bill is attributable to which patterns, or how much user-perceived latency racing providers removes) is future work.

A further direction is to treat the catalog as the seed of a *pattern language* rather than a flat list. The patterns are not independent: they compose (pre-computation feeds *Prompt Amortization*; *LLM Output Provenance* records how an artifact was made and *LLM Content Validation Tracking* whether it has since been confirmed) and they evolve over a system's lifetime (an LLM may begin as a *Rent, Then Build* stand-in, hand off to a specialized tool, and remain as the *Escalate to the LLM* fallback). Documenting these interactions, sequences, and tensions, so a designer can navigate from one pattern to the next, is a contribution beyond the individual patterns.

These choices are not made once. A system's pattern mix is actively revised as it scales and the provider landscape shifts: Zeeguu began by making single LLM calls and only recently added parallel, raced ones, and may later drop that approach or fold it together with per-user budgeting as usage grows. A longitudinal account of how and why a system's chosen patterns change over time would deepen the lifecycle dimension that this single-snapshot catalog can only sketch.

Finally, we expect the catalog to be incomplete[40]: further patterns will likely surface only through engagement with practitioners working at different scales, in different domains, and with different LLM providers.

10 Conclusion

This paper has presented a catalog of architectural patterns for integrating LLMs as components into existing user-facing applications: a setting that has received much less attention than the design of LLM-native products, despite arguably affecting a larger share of working software engineers. The patterns address recurring concerns around cost, latency, lifecycle, data management, and quality assurance, and were drawn from real production experience on the Zeeguu platform. They are intended as a starting point for community refinement rather than a closed taxonomy:

[40] The patterns in this paper are a selection from a larger, evolving catalog maintained online at https://mircealungu.github.io/llm-integration-patterns/

practitioners working on LLM integration in other domains are warmly invited to validate, refute, extend, or replace individual patterns, and to contribute new ones the catalog does not yet contain.

11 Acknowledgments

We thank Viktor Matovič, our shepherd at PLoP, for feedback that strengthened the paper throughout, and above all for his insistence that patterns be observed rather than asserted, which improved our methodology and ultimately changed the catalog itself. We thank Souhaila Serbout, Oscar Nierstrasz, and Diomidis Spinellis for feedback and discussions on earlier versions of this work. We are also grateful to the participants of the EuroPLoP focus group, whose discussion sharpened several patterns, from their naming to their relationship with both older, non-LLM patterns and the emerging LLM-gateway infrastructure.

References

[1] Andreessen Horowitz. 2023. Emerging Architectures for LLM Applications. https://a16z.com/emerging-architectures-for-llm-applications/

[2] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research (TMLR)* (2023). https://arxiv.org/abs/2305.05176

[3] Zhoujun Cheng, Jungo Kasai, and Tao Yu. 2023. Batch Prompting: Efficient Inference with Large Language Model APIs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track (EMNLP)*. https://arxiv.org/abs/2301.08721

[4] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. https://arxiv.org/abs/2310.03533

[5] Martin Fowler et al. 2024. Emerging Patterns in Building GenAI Products. https://martinfowler.com/articles/gen-ai-patterns/

[6] Vahid Garousi, Michael Felderer, and Mika V. Mäntylä. 2019. Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Information and Software Technology* 106 (2019), 101–121. doi:10.1016/j.infsof.2018.09.006

[7] Zelalem Gero, Chandan Singh, Hao Cheng, Tristan Naumann, Michel Galley, Jianfeng Gao, and Hoifung Poon. 2023. Self-Verification Improves Few-Shot Clinical Information Extraction. In *ICML 3rd Workshop on Interpretable Machine Learning in Healthcare (IMLH)*. https://arxiv.org/abs/2306.00024

[8] Alex Gilmore. 2007. Authentic materials and authenticity in foreign language learning. *Language Teaching* 40, 2 (2007), 97–118. https://doi.org/10.1017/S0261444807004144

[9] Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes. In *Findings of the Association for Computational Linguistics (ACL)*. https://arxiv.org/abs/2305.02301

[10] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2023. Large Language Models Cannot Self-Correct Reasoning Yet. https://arxiv.org/abs/2310.01798

[11] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. https://arxiv.org/abs/2004.12832

[12] Stephen D. Krashen. 1982. *Principles and Practice in Second Language Acquisition*. Pergamon Press. https://sdkrashen.com/content/books/principles_and_practice.pdf

[13] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://arxiv.org/abs/2303.16634

[14] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2303.17651

[15] Leon Moonen. 2001. Generating Robust Parsers Using Island Grammars. In *Proceedings of the Eighth Working Conference on Reverse Engineering (WCRE)*. 13–22. https://doi.org/10.1109/WCRE.2001.957806

[16] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2024. RouteLLM: Learning to Route LLMs with Preference Data. *arXiv preprint arXiv:2406.18665* (2024). https://arxiv.org/abs/2406.18665

[17] Peng Qi, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D. Manning. 2020. Stanza: A Python Natural Language Processing Toolkit for Many Human Languages. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL): System Demonstrations*. https://arxiv.org/abs/2003.07082

[18] Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. 2017. Snorkel: Rapid Training Data Creation with Weak Supervision. *Proceedings of the VLDB Endowment* 11, 3 (2017). https://arxiv.org/abs/1711.10160

[19] William Saunders, Catherine Yeh, Jeff Wu, Steven Bills, Long Ouyang, Jonathan Ward, and Jan Leike. 2022. Self-critiquing models for assisting human evaluators. https://arxiv.org/abs/2206.05802

[20] Larissa Schmid et al. 2025. Software Architecture Meets LLMs: A Systematic Literature Review. (2025). https://arxiv.org/abs/2505.16697

[21] Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. 2024. On the Self-Verification Limitations of Large Language Models on Reasoning and Planning Tasks. https://arxiv.org/abs/2402.08115

[22] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA Model. https://github.com/tatsu-lab/stanford_alpaca. https://github.com/tatsu-lab/stanford_alpaca

[23] Eugene Yan. 2023. Patterns for Building LLM-based Systems and Products. https://eugeneyan.com/writing/llm-patterns/

[24] Quanjun Zhang, Chunrong Fang, Yang Xie, Yuxiang Zhang, Yang Yang, Weisong Sun, Shengcheng Yu, and Zhenyu Chen. 2024. A Survey on Large Language Models for Software Engineering. (2024). https://arxiv.org/abs/2312.15223

[25] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2306.05685